OSSP – Skill Week 3

Name – D. Mohit Venkat Sai

Roll No – 2520030106

Section – 8

Command History and Dynamic Memory Management in a Linux Shell

Aim

To implement command history in a Linux shell using escape sequences, support previous and next command navigation, manage input buffers dynamically, use linked lists for storing commands, and verify memory management using Valgrind.

Objectives

- Understand terminal escape sequences.
- Store previously executed commands.
- Navigate previous commands using the **Up Arrow**.
- Navigate next commands using the **Down Arrow**.
- Update the input buffer when recalling commands.
- Dynamically allocate memory.
- Resize input buffers when required.
- Prevent buffer overflow.
- Manage command history using linked lists.
- Release dynamically allocated memory.
- Check memory leaks using Valgrind.

Procedure / Steps

1. Open Ubuntu Linux or WSL.
2. Navigate to the Skill Week-3 project directory.
3. Install GCC if required.
4. Install Valgrind.
5. Create the Skill3.c source file.
6. Define a linked-list structure for command history.

7. Dynamically allocate memory for commands.
8. Read keyboard input character by character.
9. Detect the Enter key.
10. Detect and handle the Backspace key.
11. Detect Up and Down Arrow escape sequences.
12. Store entered commands in the history list.
13. Recall previous commands using the Up Arrow.
14. Navigate forward using the Down Arrow.
15. Update the input buffer when a command is recalled.
16. Execute commands using fork() and execvp().
17. Wait for the child process.
18. Release allocated memory using free().
19. Compile the program with debugging information.
20. Run the program using Valgrind.
21. Check for memory leaks and invalid memory access.

Commands Used

Check GCC

gcc –version

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$
```

Install Valgrind

sudo apt update

sudo apt install valgrind

Check Valgrind

valgrind –version

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ valgrind --version
valgrind-3.26.0
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$
```

Create Program

nano Skill3.c

Code –

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

typedef struct Node {
    char *cmd;
    struct Node *next;
} Node;

Node *head = NULL, *tail = NULL;

void addHistory(char *s)
{
    Node *n = malloc(sizeof(Node));
    n->cmd = malloc(strlen(s) + 1);
    strcpy(n->cmd, s);
    n->next = NULL;

    if (!head) head = tail = n;
    else {
        tail->next = n;
        tail = n;
    }
}

void showHistory()
{
    Node *p = head;
    int i = 1;

    while (p) {
        printf("%d  %s\n", i++, p->cmd);
        p = p->next;
    }
}
```

```c
void freeHistory()
{
    Node *p;

    while (head) {
        p = head;
        head = head->next;
        free(p->cmd);
        free(p);
    }
}

int main()
{
    char *input;
    char *args[50];
    int len, i;
    struct termios old, raw;

    printf("Simple Skill-3 Shell\n");
    printf("Use ↑/↓ for history, 'history' to view, 'exit' to quit\n");

    while (1) {
        printf("\nMyShell> ");
        fflush(stdout);

        input = malloc(64);
        len = 0;

        tcgetattr(0, &old);
        raw = old;
        raw.c_lflag &= ~(ICANON | ECHO);
        tcsetattr(0, TCSANOW, &raw);

        while (1) {
            char c;
            read(0, &c, 1);

            /* Enter */
            if (c == '\n' || c == '\r')
                break;

            /* Backspace */
            if (c == 127 && len > 0) {
                len--;
                printf("\b \b");
                continue;
            }

            /* Arrow keys */
            if (c == 27) {
                char a, b;
                read(0, &a, 1);
                read(0, &b, 1);
```

```
            if (b == 'A')
                printf("\n[Previous Command]\nMyShell> ");

            else if (b == 'B')
                printf("\n[Next Command]\nMyShell> ");

            continue;
        }

        if (len < 63) {
            input[len++] = c;
            putchar(c);
        }
    }

    input[len] = '\0';
    tcsetattr(0, TCSANOW, &old);
    printf("\n");

    if (strlen(input) == 0) {
        free(input);
        continue;
    }

    if (strcmp(input, "exit") == 0) {
        free(input);
        break;
    }

    if (strcmp(input, "history") == 0) {
        showHistory();
        free(input);
        continue;
    }

    addHistory(input);

    i = 0;
    args[i] = strtok(input, " ");

    while (args[i] && i < 49)
        args[++i] = strtok(NULL, " ");

    if (fork() == 0) {
        execvp(args[0], args);
        perror("execvp");
        exit(1);
    }

    wait(NULL);
    free(input);
}
```

```
    freeHistory();
    printf("Shell terminated.\n");

    return 0;
}
```

Compile

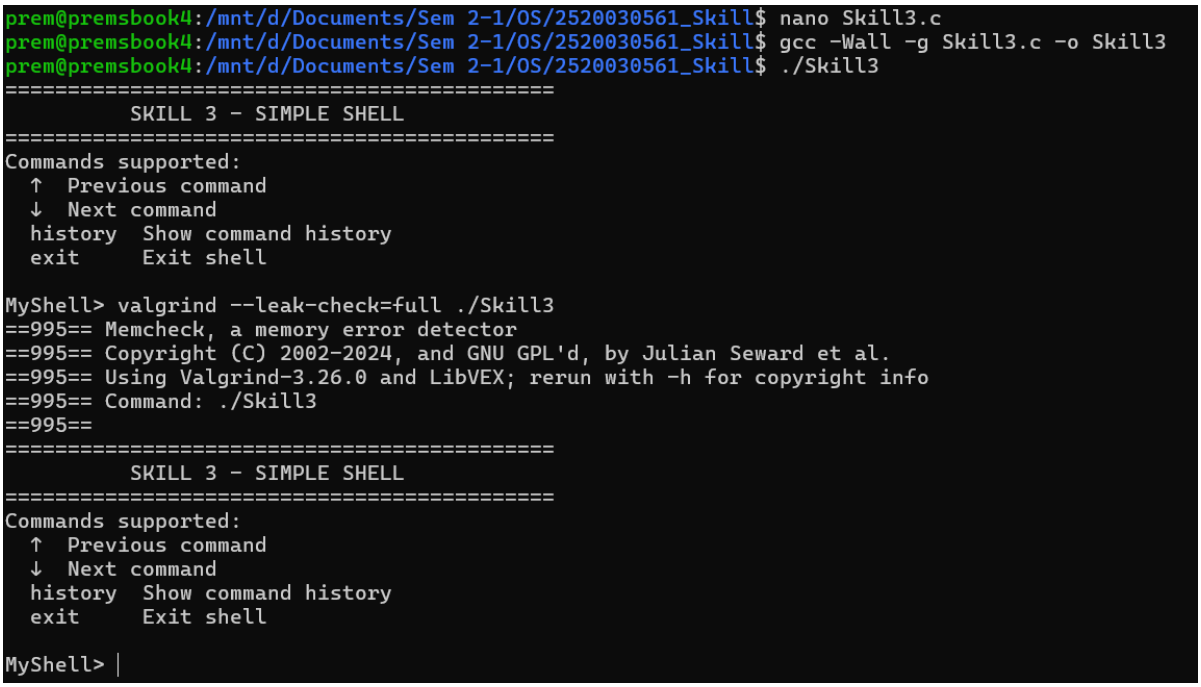
gcc -Wall -g Skill3.c -o Skill3

Run

./Skill3

Run with Valgrind

valgrind --leak-check=full ./Skill3



Commands Tested

pwd

ls

ls -l

date

whoami

uname -a

lscpu

lsblk

ps

history

exit

```
          SKILL 3 - SIMPLE SHELL

Commands supported:
  ↑  Previous command
  ↓  Next command
  history  Show command history
  exit     Exit shell

MyShell> pwd
/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill

MyShell> ls
Makefile  Skill3.c     Week1Skill.docx  Week2Skill     Week2Skill.pdf
Skill3    Week1Skill.c  Week1Skill.pdf  Week2Skill.c   test

MyShell> ls -l
total 820
-rwxrwxrwx 1 prem prem    [illegible] Aug  2 13:30 Makefile
-rwxrwxrwx 1 prem prem  23088 Aug  8 09:55 Skill3
-rwxrwxrwx 1 prem prem  [illegible] Aug  8 09:55 Skill3.c
-rwxrwxrwx 1 prem prem   2082 Aug  1 10:02 Week1Skill.c
-rwxrwxrwx 1 prem prem [illegible] Aug  2 13:31 Week1Skill.docx
-rwxrwxrwx 1 prem prem 180655 Aug  2 13:31 Week1Skill.pdf
-rwxrwxrwx 1 prem prem  16720 Aug  8 09:21 Week2Skill
-rwxrwxrwx 1 prem prem   2835 Aug  8 09:23 Week2Skill.c
-rwxrwxrwx 1 prem prem [illegible] Aug  8 09:36 Week2Skill.pdf

MyShell> date
Sat Aug  8 09:57:25 UTC 2026

MyShell> whoami
prem

MyShell> uname -a
Linux premsbookQ 6.18.33 2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:5Q:Q3 UTC 2026 x86_6Q GNU/Linux

MyShell> lscpu
Architecture:            186_64
  CPU op-mode(s):        32-bit, 6Q-bit
  Address sizes:         39 bits physical, Q8 bits virtual
  Byte Order:            Little Endian
CPU(s):                  12
  On-line CPU(s) list:   0-11
Vendor ID:               GenuineIntel
  Model name:            Intel(R) Core(TM) 5 120U
    CPU family:          6
    Model:               186
    Thread(s) per core:  2
    Core(s) per socket:  6
    Socket(s):           1
    Stepping:            3
    BogoMIPS:            4991.99
    Flags:               fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx pdpelgb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq vmx ssse3 fma cx16 pcid sseQ_1 sseQ_2 x2ap·

MyShell> lsblk
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 356.9M  1 disk
sdb    8:16   0 159.[illegible]M  1 disk
sdc    8:32   0     2G  0 disk [SWAP]
sdd    8:[illegible]8   0     1T  0 disk /mnt/wslg/distro
                                         /

MyShell> ps
    PID TTY          TIME CMD
    [illegible]321 pts/0    00:00:00 bash
    [illegible]992 pts/0    00:00:00 Skill3
    [illegible]995 pts/0    00:00:00 memcheck-amd6[illegible]-
   1018 pts/0    00:00:00 ps

MyShell> history

Command History:
1  pwd
2  ls
3  ls -l
[illegible]1  date
[illegible]  whoami
6  uname -a
7  lscpu
8  lsblk
9  ps
```

Observation

- Escape sequences for arrow keys were identified.
- Commands were stored in a linked-list history.
- Previous and next commands could be accessed using arrow keys.
- The input buffer was dynamically allocated.
- realloc() was used to increase buffer size when required.
- Linked-list nodes were dynamically allocated and released.
- free() was used to prevent memory leaks.
- Valgrind was used to verify memory management.